Tutorial Sheet — TD 2

Recurrences and the Analysis of Recursive Algorithms

Corresponds to: Lecture 3

Learning Objectives

- Translate a recursive algorithm into a recurrence relation.
- Apply the Master Theorem to solve divide-and-conquer recurrences.
- Recognize when a recurrence falls outside the Master Theorem's scope.

Exercise 1 — Solved in class

Solve the recurrence $\mathbf{T(n) = 2T(n/2) + n}$, with $\mathbf{T(1) = 1}$, using the Master Theorem.

Exercise 2 — Solved in class

Solve the recurrence $\mathbf{T(n) = 3T(n/4) + n^2}$, with $\mathbf{T(1) = 1}$, using the Master Theorem.

Exercise 3 — For the student (model answer provided below)

Consider the naive recursive Fibonacci algorithm: $\mathbf{fib(n) = fib(n-1) + fib(n-2)}$, **with** $\mathbf{fib(0) = fib(1) = 1}$.

1. Write the recurrence for its running time $\mathbf{T(n)}$, and explain briefly why the Master Theorem cannot be applied directly to it.
2. Using a recursion-tree argument, show that $\mathbf{T(n) = O(2^n)}$.
3. Describe an iterative (bottom-up) algorithm computing $\mathbf{fib(n)}$ in $\mathbf{\Theta(n)}$ time, and explain in one sentence why it avoids the exponential blow-up of the recursive version.